Information Retrieval System

Milestone 3: LLM-Augmented Retrieval

Qualitative Analysis Report

Strategies: Query Rewriting + Result Summarization

LLM Provider: Groq API | Model: llama-3.1-8b-instant

Abdullah Al Dahleh (20220385)

Tala Beirouti (2023450)

1. Introduction

This report presents the qualitative analysis for Milestone 3 of the Information Retrieval course project. Building on the classical retrieval system from Milestone 2 (VSM, BM25, and Embedding-based models), this milestone integrates a Large Language Model into the retrieval pipeline through two augmentation strategies: Query Rewriting and Result Summarization.

The LLM used is llama-3.1-8b-instant, served via the free-tier Groq API using an OpenAI-compatible interface. The API key is stored exclusively in environment variables and never hardcoded. The system was evaluated on the same 20 queries from Milestone 2, and results were compared against the Milestone 2 baseline using Precision@10, Recall, and MAP.

The evaluation produced a nuanced and instructive outcome: LLM augmentation significantly improved VSM, had no meaningful effect on the Embedding model, and unexpectedly degraded BM25 performance. Each outcome is analyzed in depth in this report.

2. LLM Augmentation Strategies Implemented

2.1 Strategy 1 — Query Rewriting

Before retrieval, the user's query is sent to the LLM with a structured system prompt instructing it to: correct any residual spelling errors, expand abbreviations, add 2–3 key technical synonyms and related terms likely to appear in relevant documents, and remove structural filler words. The rewritten query is then passed to the retrieval model in place of the original.

Temperature is set to 0.2 (near-deterministic) for reproducibility. Results are cached by input hash to avoid duplicate API calls across evaluation runs. The rewritten query and a short explanation of changes are displayed in the GUI as an expandable panel.

2.2 Strategy 2 — Result Summarization

After the top-10 documents are retrieved, their titles and the first 250 characters of their content are sent to the LLM together with the original query. The LLM returns a 3–4 sentence thematic summary of what the result set collectively covers, along with a list of the main technical themes detected across the documents.

Result Summarization is a post-retrieval augmentation: it does not affect document rankings, Precision, Recall, or MAP. Its value is entirely in user experience, helping users understand a diverse or ambiguous result set without reading all 10 documents individually.

3. Quantitative Evaluation Results

3.1 Baseline vs. LLM-Augmented Performance

Table 1 presents the full comparison across all three models and both conditions. The Δ column shows the change attributed to LLM Query Rewriting. Color coding: green = improvement, red = degradation, grey = no change.

Table 1: Baseline (M2) vs. LLM-Augmented (M3) Performance on 20 Queries

Model	Mode	Precision@10	Δ P@10	Recall	Δ Recall	MAP	Δ MAP
VSM	Baseline (M2)	0.780	—	0.0812	—	0.0694	—
VSM	**LLM-Augmented (M3)**	**0.835**	**+0.055**	**0.0907**	**+0.0095**	**0.0798**	**+0.0104**
BM25	Baseline (M2)	0.830	—	0.0895	—	0.0826	—
BM25	**LLM-Augmented (M3)**	**0.785**	**−0.045**	**0.0843**	**−0.0052**	**0.0736**	**−0.0090**
Embedding	Baseline (M2)	0.865	—	0.0935	—	0.0892	—
Embedding	LLM-Augmented (M3)	0.865	0.000	0.0956	+0.0021	0.0915	+0.0023

3.2 Summary of Findings

Model	Effect of LLM	Direction	Largest Impact Metric
VSM	Significant improvement	↑ Positive	Precision@10: +0.055 (+7.0%)
BM25	Significant degradation	↓ Negative	Precision@10: −0.045 (−5.4%)
Embedding	Negligible / marginal positive	→ Neutral	MAP: +0.0023 (+0.3%)

The most interesting finding is that the same LLM rewrites improved VSM while degrading BM25 simultaneously. This is not a contradiction — it reveals a fundamental architectural difference in how the two models process query terms, explained in Section 5

4. Qualitative Analysis

The following four cases were identified through detailed inspection of individual query results before and after LLM augmentation. They were selected to best illustrate why LLM rewriting succeeded or failed for specific model-query combinations.

4.1 Success Case 1 — VSM Recovers on Vocabulary-Poor Natural Language Queries

✔ SUCCESS CASE
Model: VSM | Query Type: Natural Language | Example: Q06, Q07, Q09, Q10

Representative example: Q06 — "How does ML improve cybersecurity threat detection?"

After LLM rewriting: "machine learning cybersecurity intrusion detection threat classification anomaly detection neural networks"

What happened: VSM's core weakness is vocabulary mismatch — it can only score documents that share exact terms with the query. The natural language queries (Q06–Q10) contain many structural words: 'How does', 'What are', 'What is the difference between', 'How can'. These words carry zero retrieval value and are absent from document vocabulary, suppressing cosine similarity scores for all relevant documents. After LLM rewriting, the query became a dense bag of high-IDF technical terms ('intrusion detection', 'anomaly detection', 'threat classification') that appear frequently in the relevant Cybersecurity and AI documents. The cosine similarity between the rewritten query vector and relevant document vectors increased substantially.

Why VSM specifically benefited: TF-IDF cosine similarity treats the query as a vector in term space. Adding more relevant terms to the query vector moves it closer to the centroid of the relevant document cluster. VSM has no penalty for longer queries — adding 5 correct terms to a 3-term query simply enriches the vector. This is why all 5 natural language queries (Q06–Q10) showed improved VSM results after LLM rewriting.

Quantitative impact: The majority of VSM's +0.055 P@10 gain is attributable to natural language and noisy queries (10 out of 20 queries), where vocabulary bridging was most needed.

4.2 Success Case 2 — VSM Handles Noisy Queries After LLM Enrichment

✔ SUCCESS CASE
Model: VSM | Query Type: Noisy/Typo | Example: Q16, Q17, Q18, Q19, Q20

Representative example: Q18 — "cybr securtiy atack prevntion"

After pyspellchecker (M2 baseline): "cyber security attack prevention"

After LLM rewriting (M3): "cybersecurity attack prevention intrusion detection malware firewall vulnerability threat mitigation"

What happened: The Milestone 2 spell corrector correctly recovered the individual words, but 'cyber security attack prevention' is still a fairly generic four-word query. Cybersecurity documents in the corpus tend to use compound technical vocabulary: 'intrusion detection system', 'malware analysis', 'vulnerability assessment', 'firewall configuration'. The LLM rewrite added exactly this vocabulary. For VSM, each added term independently contributed to the cosine similarity score with relevant documents. Documents that contained 'intrusion detection' and 'malware' simultaneously now scored much higher than documents that only had 'attack' or 'prevention'.

Why this is a genuine improvement: The LLM correctly identified that a query about 'cyber attack prevention' maps to a specific cluster of Cybersecurity documents and added the precise technical vocabulary of that cluster. This is effective corpus-aware query expansion — even though the LLM has no direct access to the corpus, its training data contains the same technical vocabulary found in Cybersecurity papers, enabling it to bridge the gap.

4.3 Failure Case 1 — LLM Expansion Overloads BM25 with Cross-Topic Terms

✘ FAILURE CASE
Model: BM25 | Query Type: All types | Overall: −0.045 P@10, −0.009 MAP

Representative example: Q11 — "network security"

After LLM rewriting: "network security protocols intrusion detection firewall encryption cybersecurity threat prevention access control"

What happened: BM25 degraded across nearly all query types after LLM rewriting. The root cause is a mathematical property of BM25 called IDF dilution. When the LLM expands a 2-word query into an 8-word query, BM25 must now score documents against all 8 terms independently. Terms like 'protocols', 'access control', and 'encryption' are not exclusive to Cybersecurity — they appear in Computer Networks, Blockchain, Cloud Computing, and IoT documents as well. BM25's IDF weight for these cross-topic terms is lower precisely because they appear in many documents across many topics. Adding these terms to the query introduced noise that pulled non-Cybersecurity documents into the top-10.

Mathematical explanation: In BM25, the score for a document d given query q is the sum of IDF(t) x f(t,d) contributions for each query term t. When query length increases from 2 terms to 8 terms, the contribution of the original 2 high-IDF specific terms ('network', 'security') is now diluted by 6 additional lower-IDF terms. Documents that happen to contain more of these 6 common terms can now outrank documents that are genuinely about 'network security' but don't use all 8 terms.

Why VSM did not suffer the same problem: TF-IDF cosine similarity normalizes by the L2 norm of both the query and document vectors. Adding more terms to the query increases the query vector's norm, which is divided out during normalization. In contrast, BM25 sums raw term scores and its length normalization (parameter b) applies to the document length, not the query length. Longer queries are not penalized in the same way — but additional cross-topic terms can still introduce false positives.

Conclusion: BM25 is sensitive to query term specificity. Adding technically correct but topically broad terms is more harmful to BM25 than to VSM, because BM25's additive scoring model accumulates noise from each additional imprecise term.

4.4 Failure Case 2 — LLM Misresolves Ambiguous Queries

✘ FAILURE CASE
Model: BM25 | Query Type: Ambiguous | Example: Q13 — "data mining"

Original query: "data mining"

After LLM rewriting: "data mining techniques machine learning algorithms pattern recognition feature extraction clustering association rules"

What happened: The LLM correctly identified 'data mining' as a Data Science concept and enriched it with accurate Data Science vocabulary. However, this query was deliberately designed to be ambiguous: its ground-truth relevant set includes BOTH Data Science documents (data mining techniques) AND Blockchain documents (cryptocurrency mining). The LLM resolved the ambiguity toward Data Science and eliminated any retrieval of Blockchain documents.

Why this hurt BM25 specifically: The rewritten query contained no terms related to Blockchain ('cryptocurrency', 'bitcoin', 'hash', 'mining reward', 'proof of work'). BM25 scores Blockchain documents near zero for this rewritten query. The ground-truth relevant set for Q13 includes 60 Blockchain documents, all of which dropped out of the results entirely. This caused a large drop in Recall for Q13 and directly contributed to BM25's overall MAP degradation.

Broader implication: The LLM has no access to the corpus or the query's ground-truth relevant set. When a query is inherently ambiguous, the LLM will resolve the ambiguity based on the most statistically common interpretation in its training data. The correct approach for ambiguous queries is not disambiguation but expansion: generating multiple candidate queries, one per plausible interpretation, and merging the result sets. Alternatively, the system could detect ambiguity and prompt the user to clarify before applying rewriting.

5. Discussion

5.1 Why the Same Rewrites Improved VSM but Degraded BM25

The most important finding of this evaluation is that identical LLM rewrites produced opposite outcomes for VSM and BM25. This is explained by the architectural difference between the two models:

Property	VSM (TF-IDF Cosine)	BM25
Score type	Normalized dot product (cosine)	Sum of per-term contributions
Query length effect	Normalized — longer queries penalized by L2 norm	Not normalized — each term adds to total score
Effect of adding relevant terms	Increases cosine sim with relevant docs	Increases score, may also boost unrelated docs
Effect of adding cross-topic terms	Small negative (slightly misaligned query vector)	Accumulates noise: can push unrelated docs above relevant ones
M3 outcome	+0.055 P@10 (improved)	−0.045 P@10 (degraded)

5.2 Why the Embedding Model Was Unaffected

The Embedding model showed essentially no change in Precision@10 (0.000 delta) and minimal improvement in Recall and MAP. This is expected: the all-MiniLM-L6-v2 Sentence Transformer already encodes semantic meaning. When the LLM adds synonyms to a query, those synonyms are already semantically close to the original query in the embedding space. The rewritten query embedding is nearly identical to the original query embedding in terms of cosine similarity with document embeddings. The Embedding model's semantic coverage made LLM vocabulary expansion redundant.

The small MAP improvement (+0.0023) reflects a marginal benefit from removing structural words ('How does', 'What are') in natural language queries. Even in the embedding space, removing irrelevant tokens provides a very slight focus improvement.

5.3 Value of Result Summarization

Result Summarization does not affect retrieval metrics but provides clear user experience value in two scenarios: (1) ambiguous queries where results span multiple topics, where the summary explains why documents from different topics appeared together; and (2) natural language questions where users expect a direct answer rather than a list of links. The theme detection feature was particularly effective on Q11 ('network security'), correctly identifying Cybersecurity, Computer Networks, and Blockchain as the three dominant themes in the returned documents.

6. Conclusion

This milestone integrated LLM Query Rewriting and Result Summarization into the Milestone 2 IR pipeline using the free Groq API (llama-3.1-8b-instant). The evaluation on 20 queries produced three distinct outcomes:

- VSM improved substantially (+0.055 P@10, +0.0104 MAP). LLM vocabulary bridging was highly effective for a model that depends entirely on exact term overlap.
- BM25 degraded (−0.045 P@10, −0.009 MAP). The same vocabulary expansion that helped VSM introduced cross-topic noise in BM25's additive scoring model. The two models respond differently to query length expansion due to architectural differences in normalization.
- Embedding was unaffected (0.000 ΔP@10). The model's built-in semantic encoding already provides the coverage that LLM expansion offers, making rewriting redundant.

The core lesson is that LLM augmentation is not universally beneficial: its effectiveness depends on the retrieval model's underlying mechanics. Query rewriting is most valuable for lexical models (VSM, BM25) on vocabulary-poor queries, but must be applied carefully for BM25 to avoid over-expansion. For ambiguous queries, the LLM's tendency to resolve ambiguity in favor of one interpretation actively hurts performance by eliminating relevant documents from other valid interpretations.

A practical recommendation for future work is a hybrid routing strategy: apply LLM rewriting only for natural language and noisy queries when using VSM; disable it for BM25 unless term specificity can be enforced; and skip it entirely for the Embedding model. For ambiguous queries, multi-hypothesis expansion (generating one rewrite per plausible interpretation) would preserve coverage across all relevant topics.

Appendix A: Full Results Breakdown

Complete Metric Comparison (Baseline M2 vs LLM-Augmented M3)

Model	Mode	P@10	ΔP@10	Recall	ΔRecall	MAP	ΔMAP
VSM	Baseline	0.780	—	0.0812	—	0.0694	—
VSM	**LLM-Augmented**	**0.835**	**+0.055**	**0.0907**	**+0.0095**	**0.0798**	**+0.0104**
BM25	Baseline	0.830	—	0.0895	—	0.0826	—
BM25	**LLM-Augmented**	**0.785**	**−0.045**	**0.0843**	**−0.0052**	**0.0736**	**−0.0090**
Embedding	Baseline	0.865	—	0.0935	—	0.0892	—
Embedding	LLM-Augmented	0.865	0.000	0.0956	+0.0021	0.0915	+0.0023

Appendix B: Summary of All Four Qualitative Cases

#	Type	Model	Query	LLM Effect	Root Cause
1	SUCCESS	VSM	Natural Language queries (Q06–Q10)	P@10 improved: structural question words replaced by dense technical vocabulary	LLM vocabulary bridging moves query vector toward relevant document cluster in TF-IDF space
2	SUCCESS	VSM	Noisy queries (Q16–Q20): e.g. 'cybr securtiy atack prevntion'	P@10 improved: LLM added 'intrusion detection', 'firewall', 'vulnerability' anchoring query to Cybersecurity	Spell-corrected words were still generic; LLM added high-IDF topic-specific terms
3	FAILURE	BM25	All query types: e.g. 'network security' → expanded to 8 terms	P@10 degraded: added cross-topic terms introduce noise via BM25's additive scoring model	BM25 sums per-term scores without L2 normalization; each added low-IDF term boosts unrelated docs
4	FAILURE	BM25	Ambiguous queries (Q11–Q15): e.g. 'data mining'	Recall degraded: LLM chose Data Science interpretation, eliminating all Blockchain relevant docs	LLM has no corpus access; resolves ambiguity based on training data frequency, not corpus distribution